DECA Lab Logbook — Spring Term

Lab 2 | Challenge: Radix-4 Booth Multiplier

Challenge: Radix-4 Booth Hardware Multiplier

1. Overview

The standard shift-and-add multiplication algorithm (as implemented in Tasks 3, 4, 5) requires up to n partial products for an n-bit multiplier, meaning 16 iterations for 16-bit numbers. Each iteration involves a conditional add and a shift, so the total clock cycle count scales linearly with the word length.

Radix-4 Booth encoding is a well-known hardware technique that reduces the number of partial products by approximately half, to n/2. This is achieved by examining three overlapping bits of the multiplier at a time rather than one. For a 16-bit multiplier, only 8 partial products are needed instead of 16, significantly reducing the number of add operations required.

This logbook entry derives the Radix-4 Booth encoding from first principles, constructs the partial product lookup table, and describes how this technique is implemented in the EEP1 architecture as a Challenge extension.

2. Two's Complement Representation of the Multiplier

Consider an n-bit two's complement number B. Its decimal value is:

$$B = -B_{n-1}2^{n-1} + \sum_{i=0}^{n-2} B_i 2^i$$

3. Algebraic Manipulation

The key mathematical trick in Booth encoding is to rewrite the sum of consecutive bits using the identity:

$$2^n A_n = 2^{n+1} A_n - 2^n A_n$$

$$2^{n-2} A_{n-2} = 2^{n-1} A_{n-2} - 2^{n-2} A_{n-2}$$

$$2^{n-4} A_{n-4} = 2^{n-3} A_{n-4} - 2^{n-4} A_{n-4}$$

Applying this substitution repeatedly across the bit sum regroups the bits of B into overlapping groups of 3, stepping 2 bits at a time.

$$\begin{aligned} A = {} & 2^{n+1} A_n - 2^n A_n + 2^{n-1} A_{n-1} + 2^{n-1} A_{n-2} - 2^{n-2} A_{n-2} \\ & + 2^{n-3} A_{n-3} + 2^{n-3} A_{n-4} - 2^{n-4} A_{n-4} + \cdots + 2A_1 + A_0 \end{aligned}$$

With further reorganisation, the sum becomes:

$$\begin{aligned} A = {} & 2^{n+1} A_n + 2^{n-1}(-2A_n + A_{n-1} + A_{n-2}) \\ & + 2^{n-3}(-2A_{n-2} + A_{n-3} + A_{n-4}) + \cdots + 2(-2A_2 + A_1 + A_0) \end{aligned}$$

4. Analysis of the Booth Digit Range

Each parenthesised term is a Booth digit d_j, defined as:

$$d_j = -2B_{2j+1} + B_{2j} + B_{2j-1}$$

Each Booth digit d_j can take values from the set {−2, −1, 0, +1, +2}. For a 16-bit multiplier, the index runs j = 0, 1, …, 7, giving exactly 8 Booth groups. The lowest group (j = 0) requires B_{-1}, which does not exist in the original word; it is defined as $B_{-1} = 0$ and handled in hardware by merging a constant '0' with bits[1:0] of the multiplier.

B_{2j+1}	B_{2j}	B_{2j-1}	$-2B_{2j+1} + B_{2j} + B_{2j-1}$	Partial Product
0	0	0	0	0
0	0	1	+1	+A
0	1	0	+1	+A
0	1	1	+2	+2A
1	0	0	−2	−2A
1	0	1	−1	−A
1	1	0	−1	−A
1	1	1	0	0

The five possible values {-2, -1, 0, +1, +2} have the following hardware implications:

- 0: No operation — add nothing.
- +A / −A: Add or subtract the multiplicand A directly.
- +2A / −2A: Add or subtract A shifted left by 1 bit.

5. Hardware Implementation for EEP1

5.0 Adder Usage

The complete EEP1 implementation uses 32 + 4 + 4 + 3 = 43 full adders, well within the 64-adder limit:

- 32 adders: the 32-bit ripple-carry adder (Adder sheet) that accumulates partial products.
- 4 adders: the 4-bit loadable counter CNT1 (Adder sheet) tracking Booth iterations.
- 4 adders: the 4-bit counter (Control Path sheet) managing instruction sequencing.
- 3 adders: the 3-bit counter CNT1 (Radix4mux sheet) selecting the current Booth-encoded group.

The final calculation completes in 9 clock cycles: 8 accumulation cycles plus one cycle of register latency.

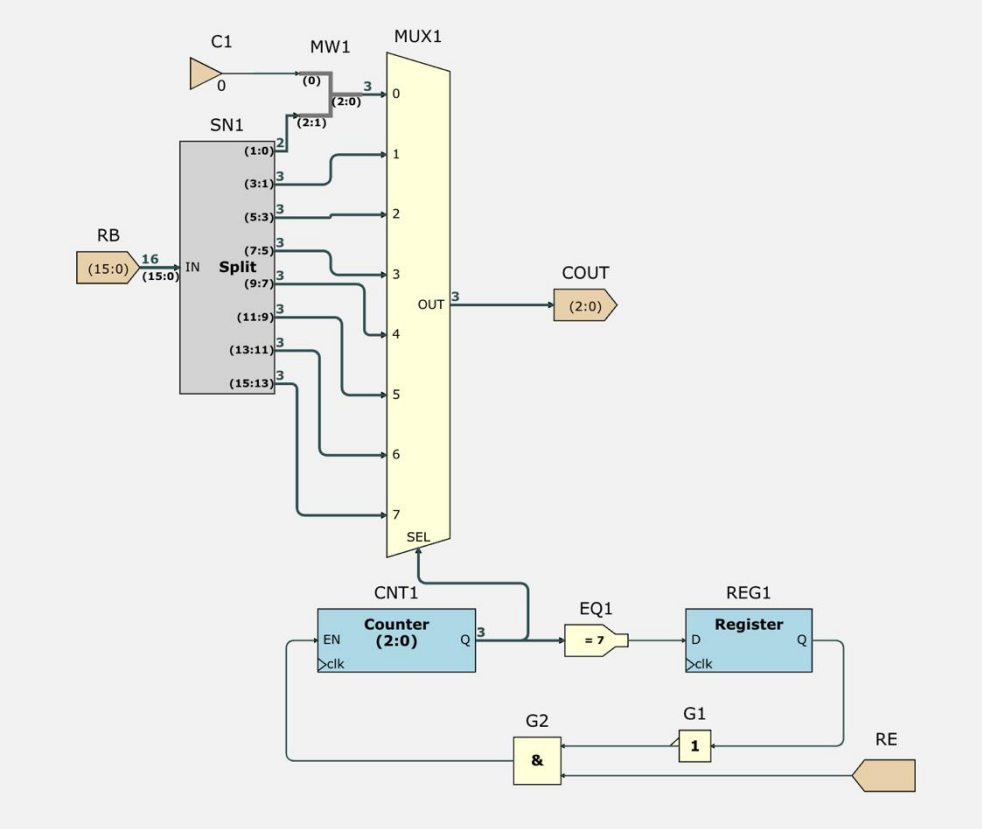
5.1 Radix4mux

The Radix4mux block selects the correct partial product at each Booth iteration. Inputs: 16-bit multiplier RB and 3-bit counter CNT1. Output: COUT(2:0), encoding the Booth digit for the current group.

RB is extended to 17 bits by prepending $B_{-1} = 0$, then split into one 2-bit group and seven 3-bit overlapping groups by SN1. The lowest group {B_1, B_0, 0} is formed by merging constant '0' (C1)

with bits[1:0] of RB via MW1. Subsequent groups cover bits [3:1], [5:3], [7:5], …, [15:13]. All eight groups feed into an 8-to-1 MUX1, with CNT1 as the select signal stepping from 0 to 7 across the 8 iterations.

Each group is 3 bits wide because Radix-4 Booth encoding requires the 2 current bits b_{2i+1}, b_{2i} and the overlap bit b_{2i-1} from the previous group. The encoded value $-2b_{2i+1} + b_{2i} + b_{2i-1}$ determines whether to add 0, ±M, or ±2M for that group.

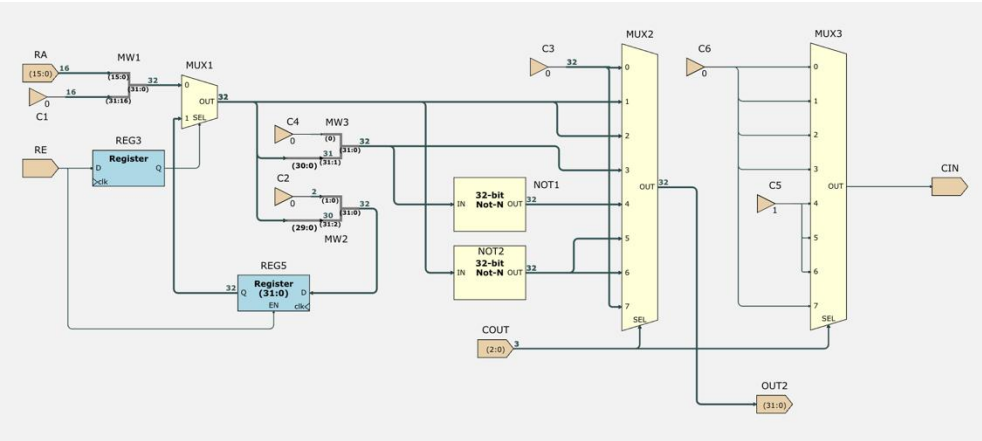


5.2 2-bit Auto Shifter (CLK2SHIFTER)

The SHIFTER block automatically shifts the multiplicand RA left by 2 bits on every clock cycle, providing the correctly scaled A (or 2A) for each successive Booth group.

On each rising edge, REG3 stores the current shifted value of RA. MUX1 selects between the original RA (first cycle, when RE just went high) and the previously stored shifted value (all subsequent cycles), ensuring the shift accumulates correctly.

To generate −A and −2A for Booth digits −1 and −2, the block applies a 32-bit bitwise NOT (NOT1/NOT2) followed by carry-in = 1, implementing two's complement negation (−X = ~X + 1). CIN = 1 whenever the Booth digit is negative ($d_j \in \{-1, -2\}$); this carry-in is forwarded to the 32-bit adder to complete the negation correctly. MUX3 selects among 0, the positive shifted value, and the NOT'd value based on COUT(2:0), routing the correct addend to the accumulator each cycle.

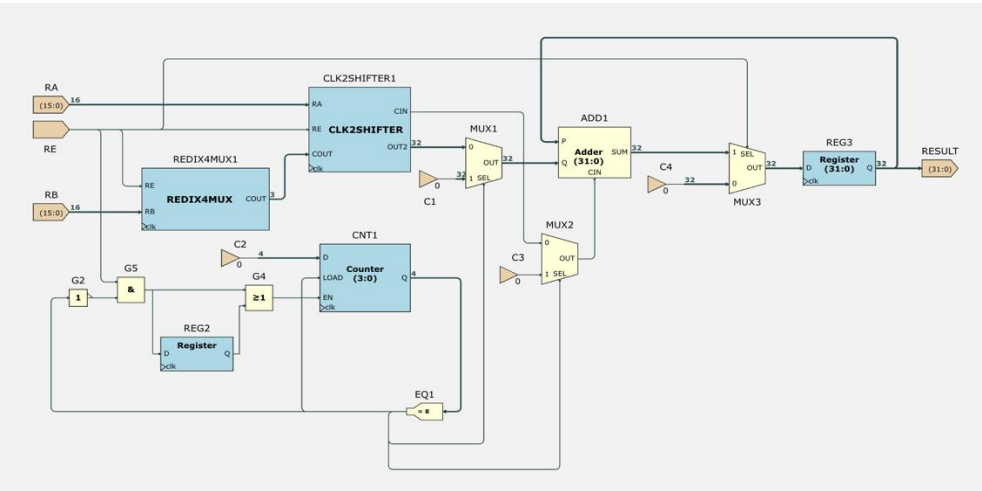


5.3 Adder

The Adder sheet orchestrates the 8-cycle Booth accumulation, instantiating CLK2SHIFTER1 and REDIX4MUX1 as sub-blocks, feeding their outputs into 32-bit adder ADD1, and accumulating the result in REG3 across all 8 iterations.

Key control logic:

- CNT1 (4-bit counter) counts the 8 Booth iterations. EQ1 fires at count = 8, disabling the addend input via MUX2 (switching to 0) to prevent a spurious final add.
- MUX3 resets REG3 to zero when RE falls, preparing the accumulator for the next multiply.
- G5 / REG2 / G4 form a one-cycle delayed feedback that keeps the counter enabled for exactly 9 cycles, matching the RE window from the control path.

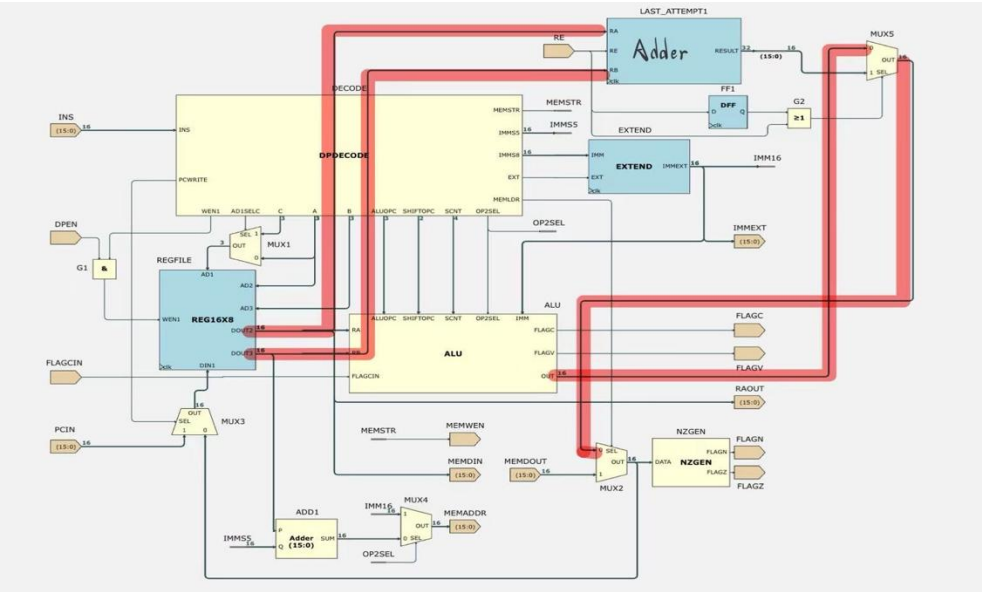


5.4 Datapath Changes

Two modifications were made to the standard EEP1 datapath:

Result register reset: An additional MUX (MUX3) controlled by RE is inserted before REG3. While RE = 1 the MUX passes the running accumulator output to REG3. When RE returns to 0 after the final cycle, MUX3 routes constant 0 into REG3, resetting it for the next multiply.

Write-back timing (DFF): A D flip-flop detects the falling edge of RE. Because RE goes low one cycle after the final accumulation completes, the DFF output is high on exactly the clock cycle when the 32-bit product is stable. Without this delay, write-back would fire while RB is still on the bus, causing the multiplier operand to overwrite the result. The DFF ensures only the correct product is written to RA.



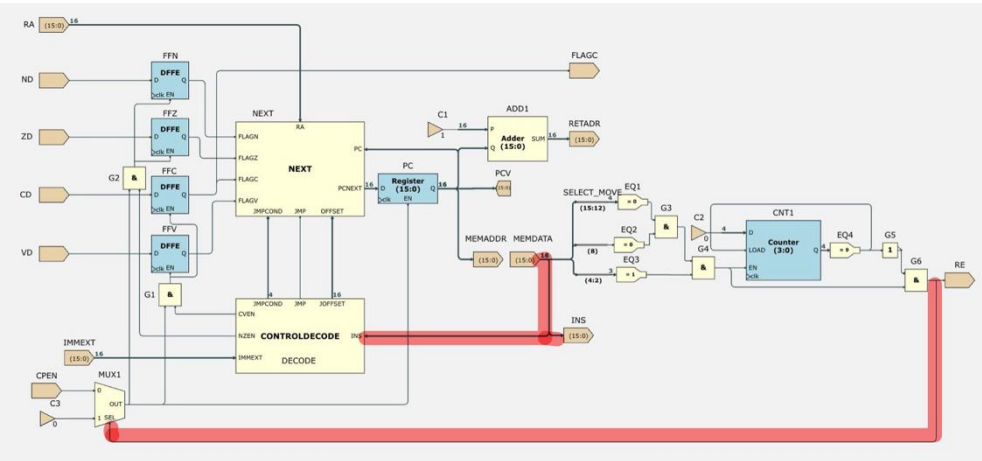
5.5 Control Path Changes

Two pieces of logic were added to generate RE and freeze the PC during multiplication:

Instruction detection: Three comparators simultaneously check INS(15:12) = 0 (MOV family), INS(8) = 0 (register mode), and INS(4:2) = 1 (MOVC1). AND gate G3 = 1 only for a MOVC1 instruction.

RE generation: When G3 fires, 4-bit counter CNT1 is loaded and begins counting. EQ4 detects count = 9 and, via NOT gate G5 and AND gate G6, drives RE high for exactly 9 cycles then drops it.

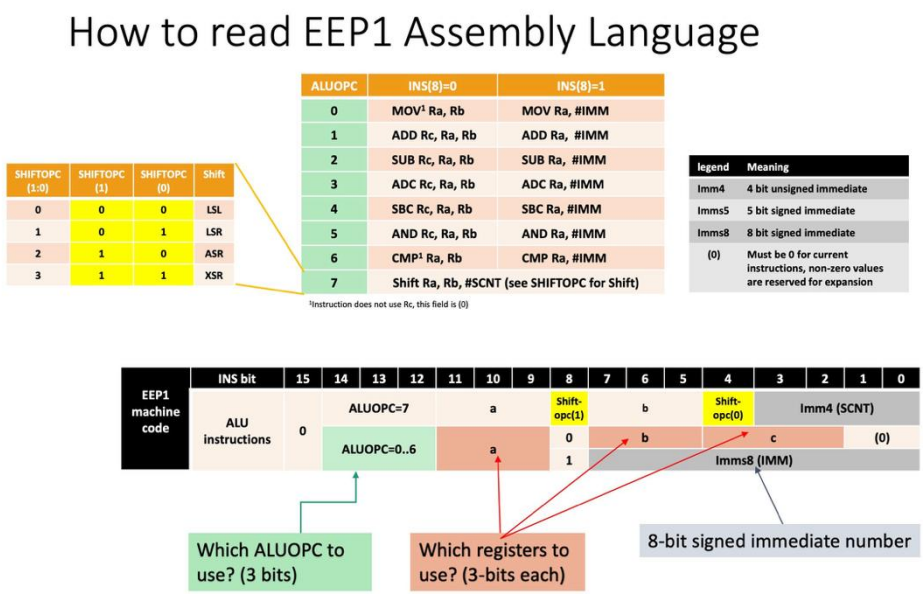
PC freeze: While RE = 1, MUX1 in the control path feeds the current PC back to PC.D, holding the program counter at the MOVC1 address. The flag flip-flop enables (CPEN) are also gated off to prevent the multiplier from corrupting the condition flags.



6. Instruction Word

The MOVC1 instruction is encoded using the unused c-field bits INS(4:2) = 1 of a standard MOV instruction. When the instruction decoder detects MOVC1:

- RE is asserted high for exactly 9 cycles.
- The PC is frozen at the MOVC1 address for those 9 cycles.
- The 32-bit Booth accumulator runs autonomously and writes the result back to RA on the falling edge of RE.



Test assembly program:

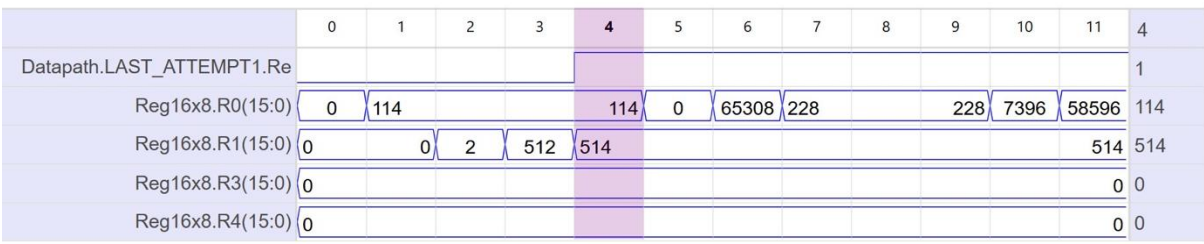
```
MOV R0, #114
MOV R1, #2
LSL R1, R1, #8      ; R1 = 512
ADD R1, #2          ; R1 = 514
MOVC1 R0, R1        ; R0 := R0 * R1 = 114 * 514 = 58,596
MOV R3, #113
LSL R3, R3, #2      ; R3 = 452
ADD R3, #127        ; R3 = 579
MOV R4, #12
MOVC1 R3, R4        ; R3 := R3 * R4 = 579 * 12 = 6,948
```

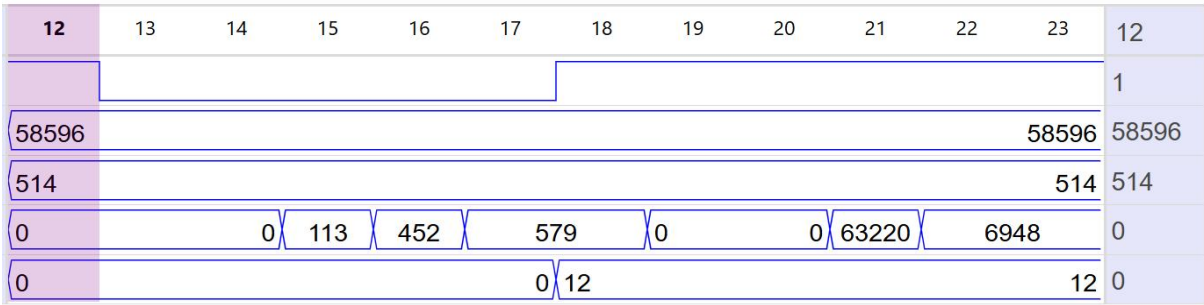
7. Simulation

The waveform simulation confirms:

- One extra cycle is taken to upload the operand value into RA before accumulation begins.
- First multiply (114 × 514): R0 receives 58,596 (0xE4A4) after 9 cycles.
- Second multiply (579 × 12): R3 receives 6,948 (0x1B24) without any manual reset, confirming that the accumulator resets correctly between consecutive MOVC1 instructions.

Total cycles per multiply: 9 (8 accumulation + 1 register latency), compared to up to 16 iterations in the baseline Task 4 software loop. This represents a 2× reduction in worst-case cycle count, achieved within 43 full adders.




12
13
14
15
16
17
18
19
20
21
22
23
12
1
58596
58596
58596
514
514
514
0
0
113
452
579
0
0
63220
6948
0
0
0
12
12
0